

A RESILIENT AND TRUST BASED OTA FRAMEWORK FOR IoT DEVICES: SECURETRUST OTA

25-26J-058

Student Name	Student ID
Dilmith K.P.H.S	IT21832130
Jayaweera N.S	IT22362780
Mandira U.L.L.K	IT22365996
P.A.S.N. Perera	IT22307262

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science
(Honours) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

We declare that this is our own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology, the non exclusive right to reproduce and distribute this dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature	Date
Dilmith K.P.H.S	IT21832130		
Jayaweera N.S	IT22362780		
Mandira U.L.L.K	IT22365996		
P.A.S.N. Perera	IT22307262		

The above candidates have carried out research for the Bachelor's degree Dissertation under my supervision.

Signature of the Supervisor:

Name: Mr. Kavinga Yapa Abeywardena

Date:

ABSTRACT

Existing Over The Air (OTA) firmware update frameworks for IoT devices suffer from static trust models, inefficient delivery mechanisms, inadequate health validation, and poor post update resilience. This dissertation presents SecureTrustOTA, a multi layered, trust based OTA framework designed for resource constrained IoT devices, targeting ESP32 microcontrollers.

The framework comprises four integrated layers. Layer 1 implements Zero Trust Authorization through a five phase adaptive authentication pipeline combining ECC P 256 cryptographic identity, hardware bound enrollment, challenge response authentication, and adaptive trust scoring. Layer 2 introduces a Lightweight Secure OTA Core that intelligently selects between delta patching and full firmware replacement using structural and complexity metrics, alongside AES 256 CCM per device encryption. Layer 3 integrates a Threshold Based Anomaly Detection Engine that monitors device telemetry which are CPU usage, memory, battery, and network latency, to validate device health before updates. Layer 4 provides a Resilient Fail Safe Update Engine with SD card staging, SHA 256 verification, a custom Finite State Machine, and hardware attestation enabling autonomous rollback within 420 milliseconds.

Evaluation demonstrates that SecureTrustOTA effectively blocks replay attacks, fingerprint tampering, signature forgery, and rollback attacks, achieving threat detection under 45 milliseconds, a 3.5 KB memory overhead, and significant bandwidth savings through delta patching.

Keywords: *IoT Security, OTA Firmware Updates, Zero Trust, Delta Patching, Anomaly Detection, Hardware Attestation, Cryptographic Security.*

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our research supervisor, Mr. Kavinga Yapa Abeywardena, for his invaluable guidance, continuous support, and expert technical insight throughout the course of this research. His direction was instrumental in shaping the SecureTrustOTA framework into a cohesive and meaningful contribution to the field of IoT security.

We also extend our appreciation to the academic and non academic staff of the Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology for providing us with an excellent learning environment and the necessary resources over the past four years.

Finally, we are grateful to each other as a team for the collective effort, shared commitment, and collaborative spirit that made this project possible.

TABLE OF CONTENTS

Contents

DECLARATION	ii
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS	v
List of Figures.....	vii
List of Tables	vii
CHAPTER 1: INTRODUCTION	viii
1.1 Background	viii
1.2 Literature Survey	ix
1.3 Research Gap	xi
1.4 Research Problem.....	xiii
1.5 Research Objectives	xiv
1.6 Research Areas	xv
CHAPTER 2: METHODOLOGY	xvii
2.1 Overall System Architecture.....	xvii
2.2 Layer 1: Zero Trust Authorization and Policy Enforcement	xviii
2.3 Layer 2: Lightweight Secure OTA Core	xx
2.4 Layer 3: Threshold Based Anomaly Detection Engine	xxi
2.5 Layer 4: Resilient Fail Safe Update Engine	xxi
2.6 Commercialization Aspects	xxvii
2.7 Testing and Implementation	xxix
Zero Trust Authorization Layer Evaluation	xxix
Lightweight Secure OTA Core Implementation	xxx
Device Side (ESP32 Firmware Implementation)	xxxi
OTA Core Testing and Performance Evaluation	xxxii
Anomaly Detection Engine Testing.....	xxxii
Resilient Fail Safe Update Engine Evaluation.....	xxxiii
CHAPTER 3: RESULTS AND DISCUSSION	xxxvi
3.1 Results	xxxvi
3.2 Research Findings	xxxvii

3.3 Discussion	xxxviii
3.4 Summary of Each Student's Contribution	xl
CHAPTER 4: CONCLUSION	xli
REFERENCES	xlii

List of Figures

Figure 1: System diagram	16
Figure 2: end to end flow of the five phase architecture	22
Figure 3: Lightweight secure OTA core	23
Figure 4: Remote edge and physical layer	24
Figure 5: Resilient OTA update architecture with physical attestation	25
Figure 6: Dashboard delta patch update	33
Figure 7: Dashboard capturing real time telemetry data	33
Figure 8: Graphical view of real time telemetry data	33
Figure 9: Alert dashboard	34

List of Tables

Table 1: Research gap	8
-----------------------------	---

CHAPTER 1: INTRODUCTION

1.1 Background

The Internet of Things is growing incredibly fast. This completely changes how technology connects billions of devices today. Right now, these setups are everywhere, from hospital monitoring systems and smart cities to basic consumer electronics. Out in the field, edge devices keep collecting and sending massive amounts of data continuously. Most of the time, these devices have to run completely on their own in places that are hard to reach physically. Because these networks keep getting bigger, keeping them secure and working properly holds huge significance. This is especially true since automated cyber threats aimed at IoT setups are getting much more advanced [1].

To handle these lifecycle problems, firmware updates are the main solution. Pushing these updates lets hardware makers and network admins patch zero day bugs or add new features. Doing this remotely prevents the impossible task of physically replacing the hardware itself. That is exactly why Over The Air update methods became a mandatory, scalable way to manage IoT devices. By sending the files over the network, OTA setups remove the need to manually touch the edge nodes. This heavily drops maintenance costs and stops unnecessary system downtime [2].

Sending these updates all the way to the network edge brings up some serious architectural limits, though. The actual hardware is the biggest constraint here. Things like the ESP32 microcontroller are heavily used in modern IoT projects, but they have very strict resource limits. They lack enough RAM, have small flash storage partitions, and run on low clock speeds [2]. Normal OTA methods usually send the complete firmware image. That takes up way too much bandwidth and easily maxes out the memory on these small devices. To fix the size issue, incremental delta patching tools like bsdiff got introduced. In theory, this works fine. But basic byte level diffing ignores how compilers actually change the firmware structure. Just a tiny code change can shift all the memory addresses. This ends up making disproportionately large patch files or causing unstable unpacking. Eventually, it just corrupts the firmware entirely [2].

Along with the delivery issues, adding security creates a totally conflicting challenge. Standard practices like transport encryption and cryptographic signatures are widely used. But they put a massive computational load on these lightweight IoT boards [3]. Trying to run a complex cryptographic handshake easily drains power reserves and triggers system timeouts. Also, most existing OTA setups wrongly assume the edge device is perfectly fine and safe before sending an update. The reality is totally different. Lots of deployed devices face silent compromises. A device might be running out of memory, facing glitches, or already controlled by an attacker. Forcing an update onto a mathematically compromised or unstable node usually ends in a complete crash. This highlights the critical significance of doing a comprehensive health check on the device before delivering any payload.

1.2 Literature Survey

A comprehensive look at existing Over The Air (OTA) frameworks shows some big improvements in secure firmware delivery. But there are still major limitations when connecting tiny edge nodes. Famous industry tools like Mender, Eclipse Hawkbit, Uptane, and The Update Framework (TUF) built some strong ways to send firmware securely. Using cryptographic encryption and strict version control, these setups successfully stop downgrade attacks. Server side rollbacks are also included. But looking at the literature, depending on static trust models is a massive, fundamental limitation. Those centralized systems only care about transporting the payload safely. Checking the actual security or working state of the receiving device before installing anything gets completely ignored [4].

In general IoT monitoring literature, using threshold based anomaly detection gets talked about a lot. Mostly, this helps to catch sensor miscalibrations or DDoS attack behaviors [5]. Even though it spots weird CPU usage, RAM allocation, and network traffic really well, there is a gap in current research. Tying these anomaly detection engines right into the OTA authorization process just hasn't happened yet. Because of that, if malicious firmware sneaks past the first signature check, it runs stealthy operations easily [5]. Things like slow memory exhaustion or unauthorized peripheral access happen without setting off any immediate system alarms.

For fixing authorization weaknesses, recent academic papers shifted towards using Zero Trust architecture. Getting rid of implicit trust and focusing on continuous mathematical verification of assets are the main principles here. Enterprise networks and cloud access controls use this with great success [6]. But applying Zero Trust specifically for OTA updates on hardware limited microcontrollers is still heavily underexplored. The existing models demand continuous authentication polling. That causes terrible latency and kills the power on edge devices running on battery reserves or narrow band networks [7].

Also, looking into post update execution resilience shows how traditional recovery methods just fail. Current edge setups mostly rely on passive hardware watchdog timers. If the main loop freezes, these timers simply force a physical reboot [3][4]. Against total system lockups or endless loops, they work fine. But watchdog timers are completely blind to logical execution errors. They also miss stealthy memory leaks and partial peripheral failures caused by newly flashed firmware. After the watchdog resets the hardware, the broken firmware just boots up again. That directly causes an infinite crash loop [8].

Finally, some research explores peer assisted mesh distribution for updates. In theory, this has huge significance for lowering centralized server bandwidth and boosting scalability. But these local distribution models create highly complex synchronization needs and nasty peer to peer trust issues. In a setup where one infected node can pass malicious patches to its neighbors, mesh based OTA needs advanced distributed consensus algorithms. For boards like the ESP32, running those is way too computationally expensive [5]. All these connected limitations covering patching efficiency, anomaly detection, Zero Trust, and fail safe recovery point to one clear thing. A unified, comprehensive, multi layered OTA framework built explicitly for resource constrained IoT edge nodes is a huge unaddressed need [9].

1.3 Research Gap

The central research gap addressed by this work is the absence of a comprehensive, integrated OTA framework that combines Zero Trust authorization, intelligent and efficient firmware delivery, lightweight anomaly based device health validation, and active hardware level post update attestation into a single, operationally viable system for resource constrained IoT devices.

Specifically, the following gaps can be identified:

- Existing OTA frameworks rely on static trust models that do not continuously re evaluate device trustworthiness per update transaction, leaving compromised or tampered devices able to receive firmware.
- Conventional delta patching operates at the byte level without structural or instruction level awareness, resulting in unstable or oversized patches when firmware is reorganized.
- No existing system integrates a decision based mechanism to intelligently evaluate whether delta patching or full image replacement is more appropriate for a given firmware transition.
- Lightweight anomaly detection methods, though suitable for constrained environments, have not been effectively integrated into OTA pre update validation workflows.
- Post update resilience mechanisms are passive and reactive, relying on watchdog timers that require complete system crashes before triggering rollback, leaving stealthy malicious payloads undetected.
- Scalability challenges in server centric OTA distribution remain unresolved, with peer assisted approaches introducing trust and synchronization difficulties.

Problem in Existing Systems	Proposed Solution in SecureTrustOTA
Static trust models do not re evaluate device trustworthiness per transaction, allowing compromised devices to receive firmware.	Layer 1 implements a Zero Trust Authorization pipeline with adaptive trust scoring that continuously re evaluates device identity and context before every update.
Byte level delta patching lacks structural and instruction level awareness, producing unstable or oversized patches after firmware reorganization.	Layer 2 employs instruction aware patch generation that operates at the structural level, producing stable and minimal patches regardless of firmware reorganization.
No existing framework intelligently decides between delta patching and full firmware replacement based on transition characteristics.	Layer 2 introduces a decision based update strategy using metrics — Structural Change Ratio, Fragmentation Score, Reconstruction Complexity, and Instability Risk — to select the optimal delivery method.
Lightweight anomaly detection has not been integrated into OTA pre update validation workflows for constrained devices.	Layer 3 integrates a Threshold Based Anomaly Detection Engine that monitors CPU, memory, battery, and network latency to validate device health before applying any update.
Post update resilience is passive, relying on watchdog timers that only trigger after full system crashes, leaving stealthy malicious payloads undetected.	Layer 4 implements an active FSM driven Fail Safe Engine with hardware attestation via a dedicated FreeRTOS task, enabling autonomous rollback within 420 ms of fault detection.
Server centric OTA distribution lacks scalability, while peer assisted approaches introduce trust and synchronization challenges.	The framework uses an air gapped SD card staging buffer to decouple distribution from device side validation, reducing server dependency while maintaining integrity.

Table 1: Research gap

1.4 Research Problem

The rapid proliferation of IoT devices has made OTA firmware updates a critical operational necessity. However, the full OTA update lifecycle, spanning device authentication, firmware delivery, pre update health validation, and post update verification, remains fragmented, insecure, and inefficient in existing systems. No single framework addresses all these stages in an integrated and cohesive manner, leaving resource constrained IoT devices persistently vulnerable across every phase of the update process.

The specific problems that exist across the OTA update lifecycle can be identified as follows:

- **Trust is evaluated only once**, at the point of initial connection, without continuous re verification throughout the update transaction. This allows compromised or tampered devices to receive firmware undetected.
- **Firmware delivery lacks structural intelligence**, as conventional delta patching operates at the byte level and produces unstable or oversized patches when firmware is reorganized, with no mechanism to decide whether patching or full replacement is more appropriate.
- **Device health is never validated before updates are applied**, meaning firmware is pushed to devices regardless of their operational state, significantly increasing the risk of failed or corrupted installations.
- **Post update resilience is passive and reactive**, relying on watchdog timers that only trigger after complete system crashes, leaving stealthy malicious payloads undetected and devices persistently compromised.

These gaps do not exist in isolation so they collectively weaken the entire OTA update pipeline. Addressing only one or two of them in isolation, as existing works tend to do, leaves the remaining attack surfaces and failure points unresolved. What is fundamentally missing is a unified, multi layered framework that enforces security, intelligence, and resilience across every stage of the OTA lifecycle as a single integrated system.

This research addresses that gap by proposing SecureTrustOTA, a four component integrated OTA framework where each component directly targets one of the above problems. Together, they form a cohesive system that enforces Zero Trust authorization, enables intelligent firmware delivery,

performs proactive device health validation, and provides active post update attestation, all within the computational constraints of resource limited ESP32 IoT environments.

1.5 Research Objectives

The main objective of this research is to design, implement, and evaluate SecureTrustOTA, a multi layered resilient and trust based OTA framework for IoT devices that enforces security, efficiency, and resilience across the full firmware update lifecycle.

The specific objectives are as follows:

- To design and implement a Zero Trust Authorization and Policy Enforcement Layer that enforces per transaction device verification through a five phase adaptive authentication pipeline combining ECC cryptographic identity, hardware bound enrollment, challenge response authentication, context bound proof, and adaptive trust scoring with dynamic RBAC and comprehensive audit logging.
- To develop a Lightweight Secure OTA Core that employs a decision based update strategy using defined metrics to intelligently select between instruction aware delta patching and full firmware replacement, integrated with AES 256 CCM per device encryption and ECC signatures.
- To integrate a Threshold Based Anomaly Detection Engine that collects and evaluates device telemetry in real time against defined operational thresholds, blocking updates to devices exhibiting anomalous behavior.
- To engineer a Resilient Fail Safe Update Engine for ESP32 microcontrollers incorporating an air gapped SD card staging buffer, streaming cryptographic verification, a custom Finite State Machine, and active hardware attestation with autonomous rollback capability.
- To evaluate the combined framework's performance in terms of security effectiveness, bandwidth efficiency, detection latency, memory overhead, and update reliability.

1.6 Research Areas

Building this unified update architecture connects a few completely different technical fields. Security, delivery, monitoring, and recovery all get combined into one working setup here. Because of that, the whole project directly pushes into four main research areas for modern IoT engineering.

Zero Trust Security in Constrained Edge Environments

Static trust is what normal network security relies on mostly. Basically, if a device joins the network, the system automatically trusts it to download files right away. This part of the research looks at bringing Zero Trust rules into hardware-limited microcontrollers. Zero Trust means completely ignoring that inherent trust. Creating dynamic authorization policies is the main goal here. These policies have to verify both the identity and the real-time health of an edge node before a firmware update even tries to start. Doing continuous authentication checks without killing the battery life holds huge significance. It also has to avoid causing massive network delays on slow connections.

Lightweight Cryptography for Secure Payload Delivery

To send firmware securely over the air, strong encryption and digital signatures are an absolute must. This stops attackers from messing with the files mid-air. But running standard cryptographic protocols eats up way too much memory and processing power. So, looking deeply into optimizing those heavy mathematical operations is what this research area does. Getting secure transport and payload verification to run inside the tiny RAM and flash storage of something like an ESP32 is the main target. Finding a solid balance between strict physical hardware limits and high-level security needs has massive significance. That makes sure the device avoids crashing or timing out while it unpacks the new update.

Behavioral Anomaly Detection on Microcontrollers

Small devices completely fail at running normal antivirus software. That makes catching active threats a huge headache. Scanning file signatures gets completely thrown out here. Instead, this

research section switches the focus straight into checking actual device behavior. Watching real-time system metrics helps a lot. Tracking sudden RAM spikes, weird network traffic, and abnormal CPU usage easily exposes stealthy malware or hidden memory leaks. Building a localized, threshold-based monitoring engine that just stays running in the background is the final objective. It absolutely needs to flag suspicious behavior accurately. But it cannot slow down the actual operational tasks of the edge node while doing it.

Autonomous System Recovery and Firmware Resilience

Updates can easily fail anyway, even if the delivery and authorization were highly secure. Network drops, sudden power loss, or totally hidden logical bugs in the new code usually cause these crashes. Making edge nodes physically resilient against those exact failures is the focus of this research area. Setting up air-gapped staging buffers gets tested here just to safely store incoming updates before flashing them. Building finite state machines to manage the whole boot process also gets covered. Creating a comprehensive, autonomous rollback capability stands as the ultimate objective. Basically, the edge device needs to catch a crash or a kernel panic right after an update finishes. Then it has to revert straight back to the last working version automatically. Human intervention is completely removed from the recovery process.

CHAPTER 2: METHODOLOGY

2.1 Overall System Architecture

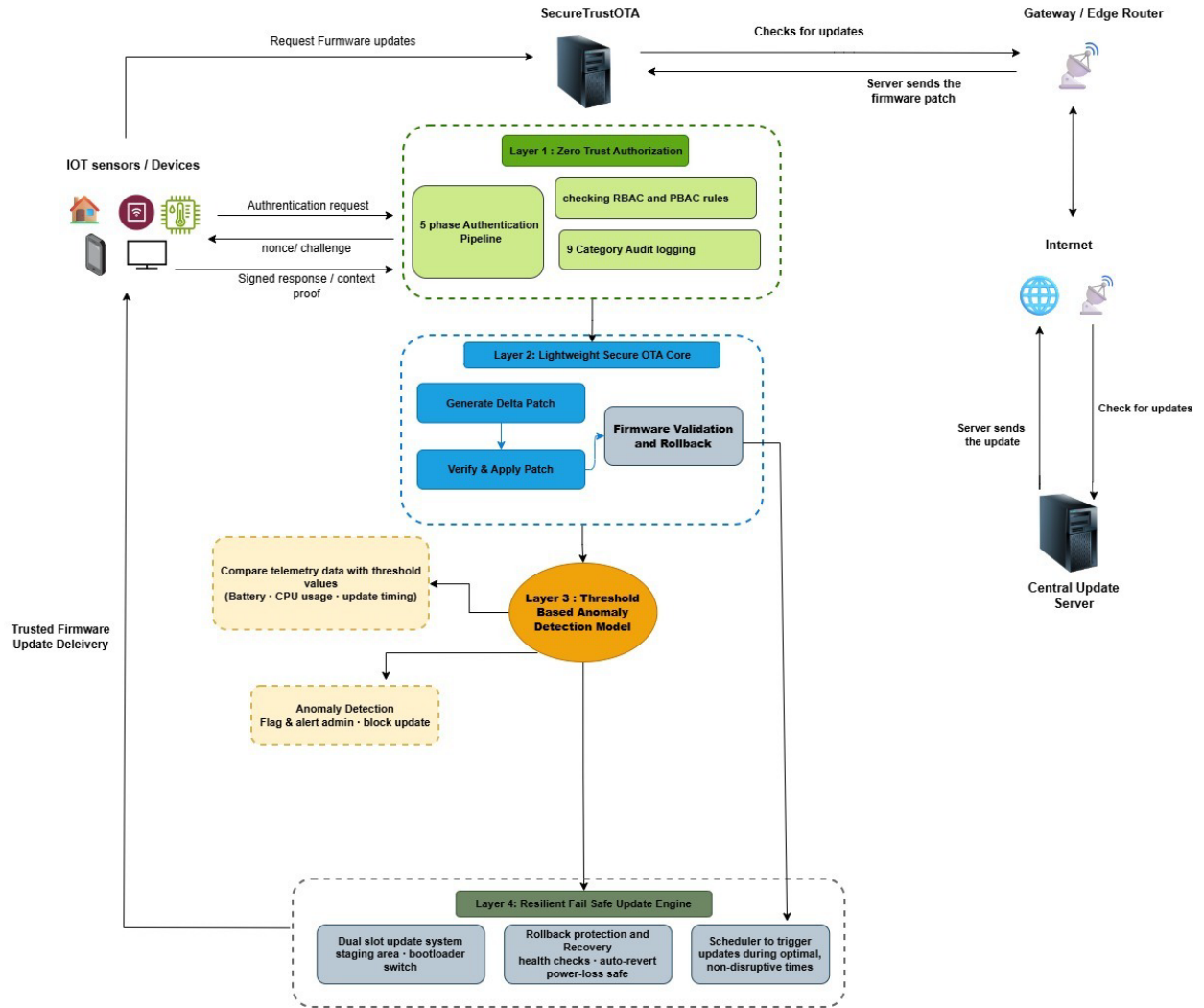


Figure 1: System diagram

SecureTrustOTA is architected as four tightly integrated and sequentially operating layers, each addressing a distinct dimension of the OTA update security problem. The architecture ensures that no firmware update is delivered to a device without passing through all layers of validation and protection.

Layer 1 – Zero Trust Authorization and Policy Enforcement Layer acts as the security gatekeeper. All firmware update requests must first pass through a five phase adaptive authentication pipeline. Only devices that complete the full pipeline and achieve a minimum adaptive trust score of 70/100 are authorized to proceed.

Layer 2 – Lightweight Secure OTA Core handles intelligent and efficient firmware delivery. Upon authorization, the server evaluates firmware differences using defined metrics and determines whether to transmit a delta patch or a full firmware image, applying instruction aware patch generation and cryptographic packaging.

Layer 3 – Threshold Based Anomaly Detection Engine performs pre update device health validation. Device telemetry is continuously compared against operational thresholds, and any device exhibiting anomalous behavior is blocked from receiving an update, with alerts generated for administrators.

Layer 4 – Resilient Fail Safe Update Engine operates directly on the ESP32 hardware after firmware installation. A custom Finite State Machine controls a probationary verification period during which a FreeRTOS based active hardware attestation task monitors system telemetry and triggers autonomous rollback if abnormal behavior is detected.

The overall flow is: device authentication through Layer 1 → intelligent firmware preparation through Layer 2 → device health validation through Layer 3 → secure delivery and post installation attestation through Layer 4.

2.2 Layer 1: Zero Trust Authorization and Policy Enforcement

The Zero Trust Authorization Layer is implemented as a Flask based server with a SQLite backend. All firmware update requests pass through this layer before reaching any downstream delivery service. The layer enforces the principle of "never trust, always verify" through a five phase progressive authentication pipeline.

Phase 1 – Baseline Cryptographic Identity: The IoT device generates an ECC P 256 key pair locally. Only the public key is transmitted to the server, which validates the key format, generates a SHA 256 fingerprint, and stores the device identity. The private key never leaves the device.

Phase 2 – Hardware Bound Enrollment: The device collects a hardware fingerprint by hashing a concatenation of its MCU unique identifier, flash storage capacity, bootloader CRC, and secure boot flag. This fingerprint is submitted and stored, binding the device's cryptographic identity to its physical hardware characteristics. Future mismatches trigger automatic quarantine.

Phase 3 – Challenge Response Authentication: The server issues a cryptographically random 256 bit nonce with a five minute expiry. The device signs the nonce using ECDSA with SHA 256 and returns the signature. The server verifies the signature, confirming proof of private key possession. Single use nonce enforcement prevents replay attacks.

Phase 4 – Context Bound Challenge Response: The device constructs a compound payload combining the nonce with its current hardware state hash, monotonic counter, and timestamp, and signs the full payload. This simultaneously proves identity and the consistency of the device's current operational context.

Phase 5 – Adaptive Trust Scoring: The server evaluates the context bound proof across three dimensions: cryptographic verification (40 points), hardware state consistency using a 5% Hamming distance drift threshold (30 points), and temporal continuity through monotonic counter and timestamp validation (30 points). Devices scoring 70/100 or above are classified as Trusted.

Following authentication, Role Based and Policy Based Access Control (RBAC/PBAC) enforcement ensures that only active, compliant devices with valid trust scores are served firmware. Nine categories of audit logs capture all security relevant events, providing full forensic traceability. Firmware attestation via SHA 256 hash verification and AES 256 CBC end to end encryption are applied to all update packages before delivery.

2.3 Layer 2: Lightweight Secure OTA Core

The Lightweight Secure OTA Core addresses the efficiency and security of firmware delivery. Upon receiving an authorized update request, the server evaluates the differences between the current and target firmware versions using four metrics to calculate a Decision Score (DS):

- **Structural Change Ratio (SCR):** measures the proportion of structural differences between firmware versions.
- **Fragmentation Score (FS):** evaluates the degree of fragmentation in the patch.
- **Reconstruction Complexity (RC):** assesses the computational cost of applying the patch.
- **Instability Risk (IR):** estimates the likelihood of patch failure given the detected structural changes.

If the Decision Score is low, delta patching is selected. If the score is high, full firmware replacement is chosen. This decision based approach avoids the inefficiency of blindly applying delta patches when structural changes render them unstable or oversized.

When delta patching is selected, an instruction aware patch generation process is employed. Rather than treating firmware as raw byte streams, the system normalizes address related values and focuses on meaningful instruction level structural differences. This produces smaller and more stable patches compared to conventional bsdiff byte level diffing, reducing bandwidth consumption and minimizing update failures.

All update packages, whether delta patches or full images, are encrypted using AES 256 CCM with per device keys, signed with ECC for authenticity, and transmitted over HTTPS. Additional features include streaming chunk based downloads to avoid large memory buffer requirements, OLED display feedback for update progress visualization, and an optional peer assisted mesh distribution model that allows devices to share updates locally, reducing server load in large deployments.

2.4 Layer 3: Threshold Based Anomaly Detection Engine

The Threshold Based Anomaly Detection Engine provides proactive pre update device health validation. The system continuously collects telemetry data from IoT devices, including CPU usage, memory utilization, battery level, network latency, and error rates, and transmits this data securely to the OTA server.

Threshold values representing acceptable operational ranges are defined based on device specifications, historical data, and operational requirements. The detection engine continuously compares incoming telemetry against these thresholds in real time. To improve accuracy, adaptive thresholding techniques adjust thresholds based on statistical analysis and historical trends.

When a device's telemetry deviates beyond defined thresholds, the system classifies the device as anomalous, generates an alert for administrators, and blocks the device from receiving any OTA update. This gating mechanism ensures that only stable and healthy devices are updated, preventing update failures and the propagation of issues to already compromised devices. Under normal conditions, the anomaly detection engine processes telemetry in under 100 milliseconds, introducing negligible overhead to the update process.

2.5 Layer 4: Resilient Fail Safe Update Engine

The Resilient Fail Safe Update Engine operates entirely on the ESP32 WROOM 32 microcontroller, providing autonomous post update resilience without requiring external cloud connectivity.

Air Gapped SD Card Staging: Incoming encrypted firmware payloads are downloaded over Wi-Fi and saved directly to an external SD card module connected via the SPI bus. The payload is not written to the internal flash memory at this stage, physically isolating the download from the primary bootable partition and preventing corruption from network interruptions or incomplete transfers.

Streaming Cryptographic Verification: Due to the ESP32's approximately 520 KB SRAM constraint, the staged firmware is read from the SD card in 4 KB blocks. Each block is

simultaneously written to the inactive Partition B in dual bank flash memory and passed through the ESP32's hardware cryptographic module, continuously updating a SHA 256 hash context. Upon completion, the final checksum is compared against the expected value in the secure file header. Any mismatch aborts the operation, leaving the primary partition untouched.

Finite State Machine and Active Hardware Attestation: After successful cryptographic verification, the device reboots into a probationary state. A custom Finite State Machine (FSM) controls a time bound probation sequence. A dedicated high priority FreeRTOS task performs Active Hardware Attestation by continuously monitoring stack memory usage, heap memory depletion rates, Wi Fi PHY matrix stability, and SPI bus connectivity.

If the FSM detects abnormal behavior such as thread starvation, rapid heap exhaustion, or unexpected peripheral disconnections, it immediately registers a fault, flags the new partition as invalid in the otadata sector, and triggers a deterministic hardware reboot. The secondary bootloader then redirects execution to the stable Partition A. This entire rollback sequence completes in 420 milliseconds, approximately 100 times faster than standard hardware watchdog timers which require 3,000–5,000 milliseconds to trigger.

The Zero Trust Authorization and Policy Enforcement Layer is implemented as a Flask based server that acts as the security gatekeeper for all OTA related device interactions. The system maintains all device state in a SQLite database with nine dedicated audit log tables, and exposes a Role Based Access Control (RBAC) verification endpoint that is called by the downstream firmware delivery server before serving any patch.

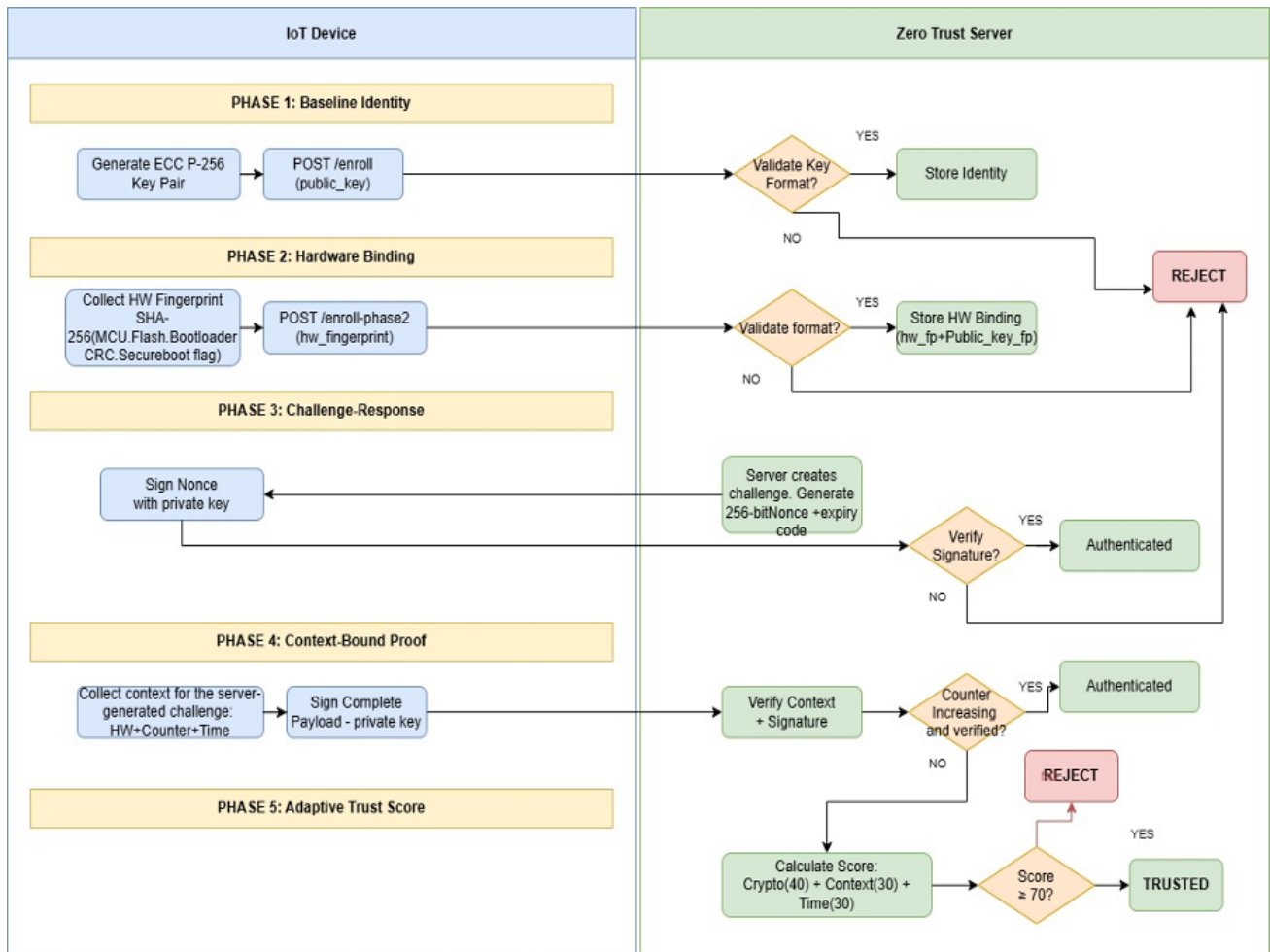


Figure : end to end flow of the five phase architecture

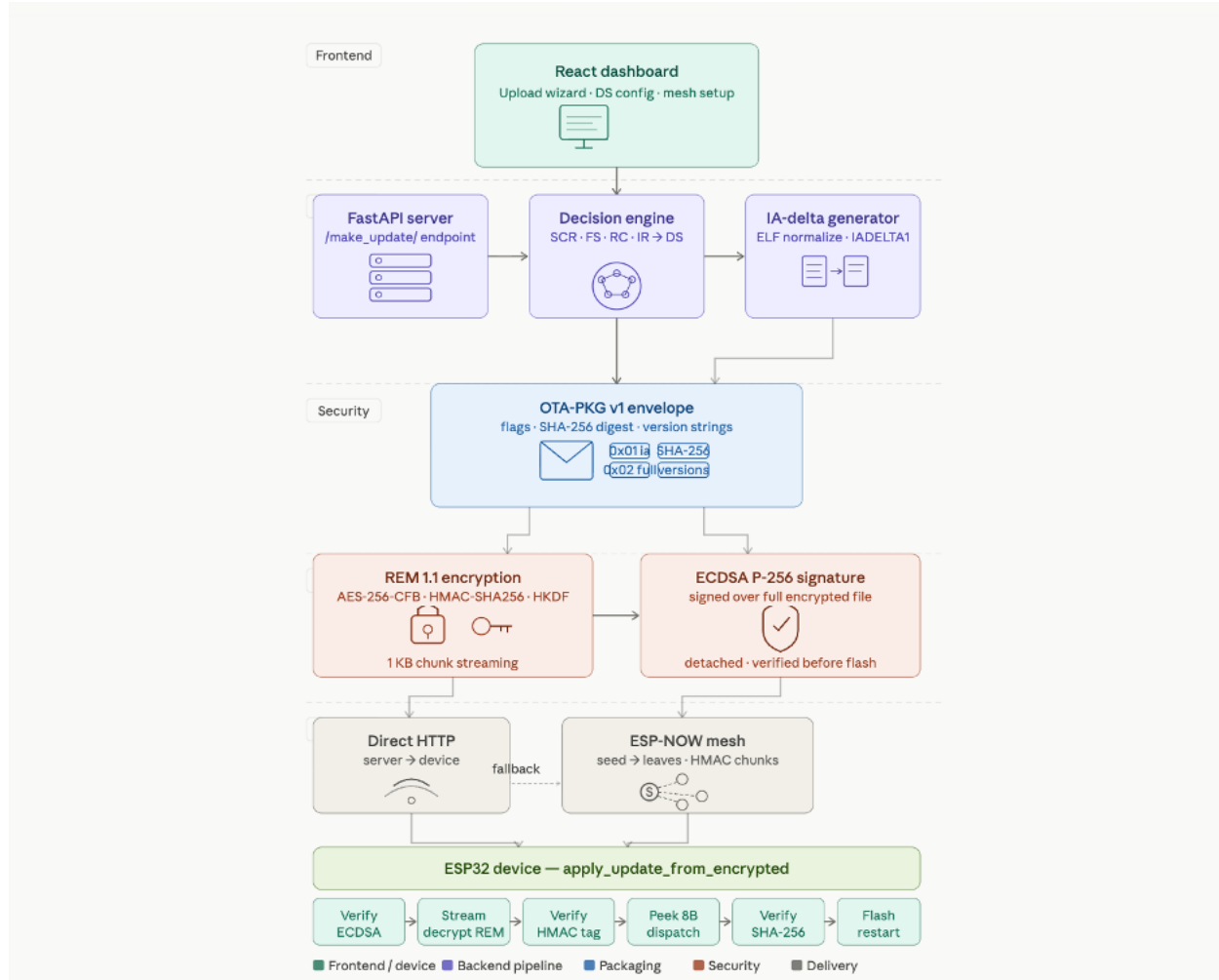


Figure: Lightweight Secure OTA Core

The Lightweight Secure OTA Core in SecureTrustOTA enables adaptive and secure firmware updates for ESP32 devices. A Decision Engine evaluates divergence metrics to select full, binary delta, or instruction aware delta updates. Instruction aware processing normalizes Xtensa instructions to improve efficiency. Updates are packaged in OTA PKG v1 with metadata for unified handling. Security is ensured using AES 256 encryption, HMAC SHA256, HKDF based key derivation, and ECDSA signatures. Distribution supports both centralized delivery and ESP NOW mesh propagation for scalability. On device, ESP32 performs streaming verification, decryption, and validation before reconstructing or flashing firmware to the inactive partition, followed by safe reboot.

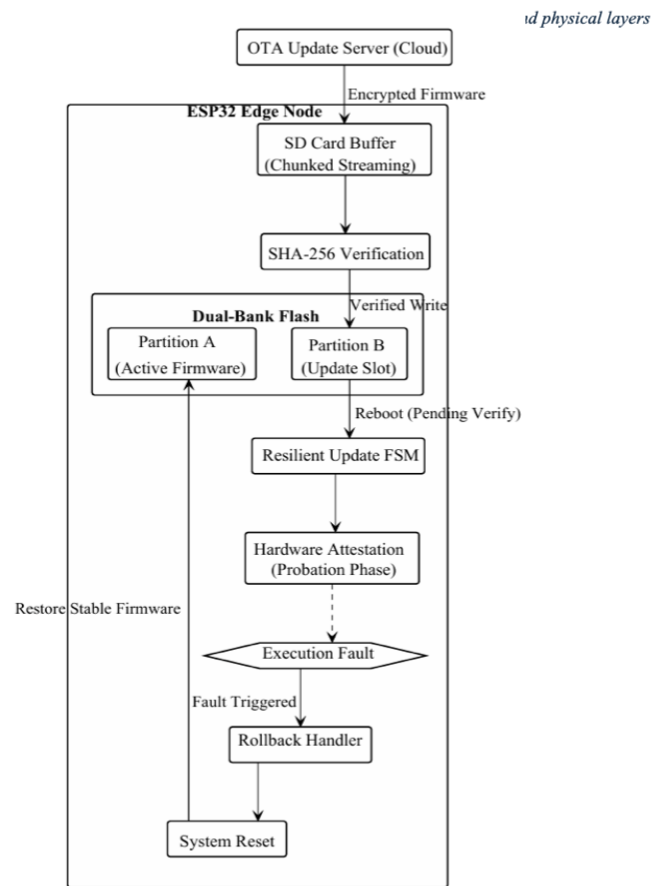


figure: Remote edge and physical layer

the fail-safe update engine gets broken down in this diagram. Pulling encrypted firmware chunks from the cloud server straight into a local SD card buffer actually starts the whole process. Because sudden network drops happen constantly, temporarily holding the data here safely prevents any memory corruption. Running a strict SHA-256 check is the immediate next step once the full download finishes. Doing this comprehensively verifies the file integrity.

Flashing the new update over to Partition B only happens after passing that strict verification. Throughout this entire step, the currently active firmware sitting on Partition A stays totally

untouched. After that, the system reboots straight into a probation phase. A Finite State Machine firmly manages this exact state. Unexpected crashes or execution faults can easily trigger during this probation window. If anything fails, the rollback handler just kicks in instantly. It forces a hardware reset and boots everything safely back from Partition A. This ultimately prevents the board from getting completely bricked.

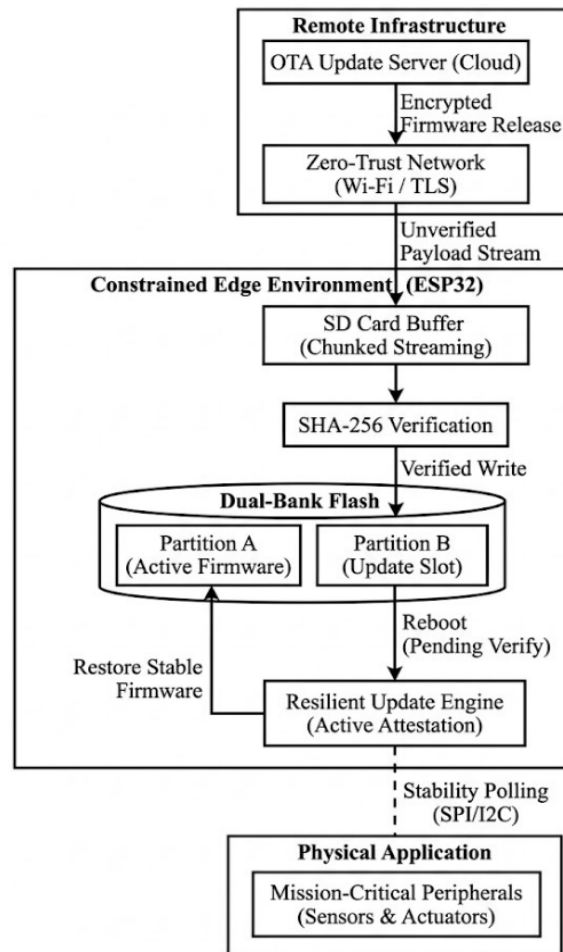


Figure: Resilient OTA update architecture with physical attestation

The full operational flow connecting the remote cloud infrastructure, the constrained edge node, and the actual physical hardware gets comprehensively mapped out in this diagram. Pushing an encrypted firmware release from the cloud server actually kicks off the whole process over in the Remote Infrastructure. Doing this securely relies heavily on a Zero-Trust network using standard Wi-Fi and TLS protocols.

Hitting the ESP32 edge environment means the unverified payload stream just dumps directly into the SD card buffer using chunked streaming. Preventing sudden memory crashes holds huge significance here, which is exactly why storing the data temporarily works so well. Running a strict SHA-256 verification happens immediately after finishing the full download. Getting a verified write straight into Partition B of the dual-bank flash only occurs if that security check passes completely. Keeping the active firmware on Partition A totally untouched and safe is the main objective during this specific step.

Handling active attestation falls entirely on the Resilient Update Engine right after a normal system reboot. That direct link directly into the Physical Application layer is definitely the most critical addition in this setup. Running comprehensive, continuous stability polling on mission-critical peripherals like sensors and actuators takes absolute priority. The update engine actively manages this using basic SPI and I2C protocols. Breaking those physical hardware connections because of newly flashed firmware gets detected by the engine almost instantly. Triggering the restore path right away is what automatically follows. This safely rolls the whole system back to the stable firmware left on Partition A to avoid a bricked board.

2.6 Commercialization Aspects

SecureTrustOTA has significant commercial potential across multiple industries where secure, reliable, and uninterrupted IoT device operation is critical.

Smart water management system for Agriculture

One practical application of this research is Smart Water Management Systems for Agriculture. In such deployments, large numbers of IoT devices are installed in fields to monitor and manage water usage. These devices are resource constrained, often solar powered, and need regular firmware updates to stay secure and functional.

Smart Homes & Consumer Devices

Everyday IoT devices, such as smart plugs, cameras, and appliances, benefit from seamless updates that guarantee device safety. Customers can confidently update their devices without the risk of malfunctions, improving user trust and satisfaction.

Smart Metering & Energy Grids

Utilities can deploy millions of smart meters with confidence, knowing that updates will not disrupt billing or regulatory compliance. Our framework ensures continuous operation and minimizes the need for costly technician interventions, helping maintain smooth energy distribution and accurate data collection.

In **healthcare**, the framework ensures that medical IoT devices such as wearable monitors and bedside sensors receive only verified, trust validated firmware updates, reducing the risk of service failures that could compromise patient safety.

In **industrial IoT and manufacturing**, the resilient update architecture enables critical security patches to be deployed to SCADA systems and factory floor devices without risk of soft bricking equipment, eliminating the need for costly physical technician interventions in remote or hazardous environments.

The framework can be offered as a cloud based SaaS product or integrated into existing IoT management platforms. For mass production, the architecture supports integration with dedicated hardware cryptographic accelerator chips such as the ATECC608A to further reduce computational overhead on constrained devices.

Commercial Value and Benefits:

- Safe upgrades: ECC signatures and AES CCM encryptions are used to guarantee integrity of the firmware and resist modifications.
- Efficient updates: Delta updates transmit only difference so that the cost of data transfer is reduced, and bandwidth is saved.
- Lightweight implementation: Is developed to support devices with a small amount of memory and energy, to reduce the processing load and power usage.

Scalable: Enables one to update hundreds of distributed sensors reliably and quickly.

Practical impact: Enables sustainable farming by ensuring the reliability of the IoT system, minimizing the maintenance effort, and increasing the life cycle of the devices.

2.7 Testing and Implementation

The SecureTrustOTA framework was comprehensively implemented and evaluated using a hybrid approach that combined both physical hardware testbeds and software based simulation environments. This dual evaluation strategy ensured that the system was tested under realistic embedded conditions while also allowing controlled experimentation with security threats and performance metrics.

Zero Trust Authorization Layer Evaluation

The Zero Trust Authorization Layer was developed as a Flask based Python server integrated with a SQLite backend for lightweight data persistence. The implementation focused on enforcing strict device authentication and preventing unauthorized firmware access.

Security validation was performed through a series of attack scenario simulations, executed against a live server deployment:

- Hardware fingerprint tampering to test device identity integrity
- Counter replay attacks to evaluate freshness validation mechanisms
- Digital signature forgery attempts to assess cryptographic robustness
- Counter rollback attacks to verify monotonic counter enforcement

All attack scenarios were successfully detected and mitigated, demonstrating the robustness of the Zero Trust model.

Lightweight Secure OTA Core Implementation

The Lightweight Secure OTA Core was tested using physical ESP32 devices, focusing on both performance and reliability across different firmware update scenarios.

Server Side (FastAPI Backend)

The backend system was designed for scalability and modularity using modern Python based technologies:

- **Python 3.1**
 - Primary implementation language for:
 - Decision Engine
 - Delta generation pipeline
 - REM 1.1 encryption layer
 - Chosen for its extensive ecosystem of binary analysis and cryptographic libraries
- **FastAPI Framework**
 - Lightweight asynchronous web framework
 - Provides REST endpoints for:
 - Firmware upload
 - Update queuing
 - Decision engine configuration (/api/decision/config)
- **Pyelftools**
 - Used to parse ELF firmware binaries
 - Extracts:
 - .text sections
 - Function symbols
 - Instruction level details
- **bsdiff4 / bsdiff**
 - Core binary differencing engine
 - Supports:
 - Standard delta generation

- Instruction aware delta patching
- **Cryptography Library (Python)**
 - Implements server side security operations:
 - AES 256 CFB encryption
 - HMAC SHA256 integrity checks
 - HKDF SHA256 key derivation
 - ECDSA P 256 digital signatures

Frontend (React Dashboard)

A user friendly interface was developed to manage OTA operations:

- **React.js**
 - Provides an interactive dashboard with:
 - Firmware upload wizard
 - Decision score visualization
 - Update strategy selection display
 - Mesh group configuration interface

Device Side (ESP32 Firmware Implementation)

The device side firmware was implemented using native embedded tools and optimized for constrained environments.

- **ESP IDF (Espressif IoT Development Framework)**
 - Provides:
 - OTA partition management
 - SPIFFS file system support
 - Hardware abstraction layer
- **mbedTLS**
 - Built in cryptographic library used for:
 - AES 256 CFB streaming decryption

- HMAC SHA256 verification
- HKDF SHA256 key derivation
- ECDSA P 256 signature verification
- **ESP NOW Protocol**
 - Connectionless peer to peer communication
 - Enables:
 - Seed device broadcasting
 - Leaf node reception
 - Mesh based OTA distribution
- **C Language (ESP IDF Component Model)**
 - All firmware written in C
 - Compiled using the Xtensa GCC toolchain
 - Optimized for low memory and power usage

OTA Core Testing and Performance Evaluation

The OTA Core was evaluated using multiple performance metrics:

- Decision score calculation accuracy
- Stability of patch generation
- Bandwidth consumption efficiency
- Reliability of updates across firmware versions with varying structural changes

Testing confirmed that the adaptive update strategy significantly reduced bandwidth usage while maintaining high reliability.

Anomaly Detection Engine Testing

The Anomaly Detection Engine was evaluated under both normal and abnormal operating conditions to ensure reliable threat detection.

Test Scenarios Included:

- High CPU usage conditions
- Low battery levels
- Network instability
- Multiple simultaneous anomalies

The system successfully identified abnormal patterns and triggered appropriate responses, validating its effectiveness in resource constrained environments.

Resilient Fail Safe Update Engine Evaluation

The Fail Safe Update Engine was tested using a physical hardware testbed, consisting of:

- ESP32 microcontroller with 4MB external SPI flash
- SD card module for firmware staging
- OLED display for runtime monitoring

Failure Injection Testing:

A synthetic threat was introduced by sending a firmware update containing a malicious **FREEZE directive**, simulating a critical execution failure during the probationary phase.

Observed Outcomes:

- Fault was successfully detected during execution
- System triggered rollback to the previous stable firmware
- Device remained operational without permanent failure

This demonstrated the effectiveness of the fail safe mechanism in ensuring system resilience and recovery.

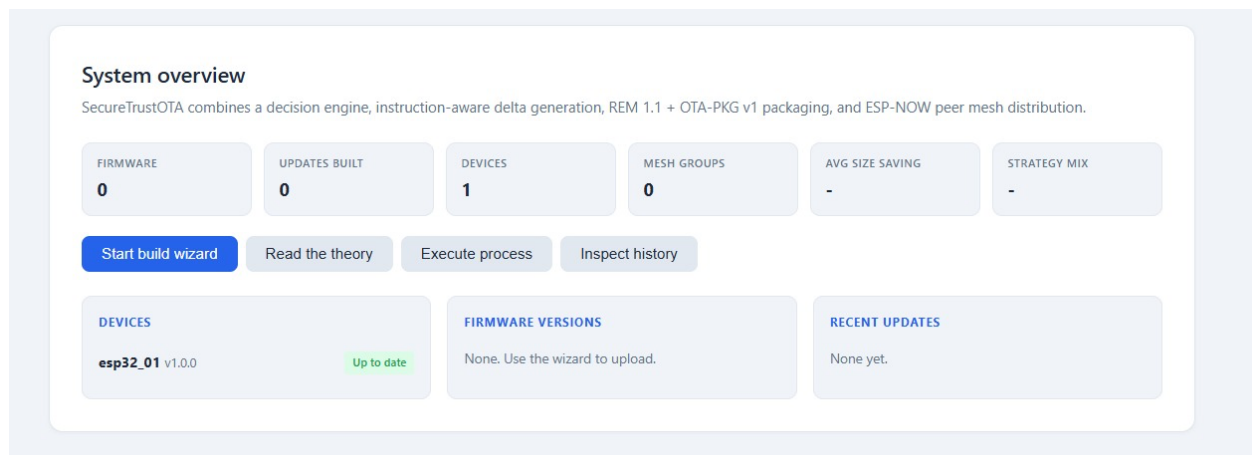


Figure: dashboard: delta patch update

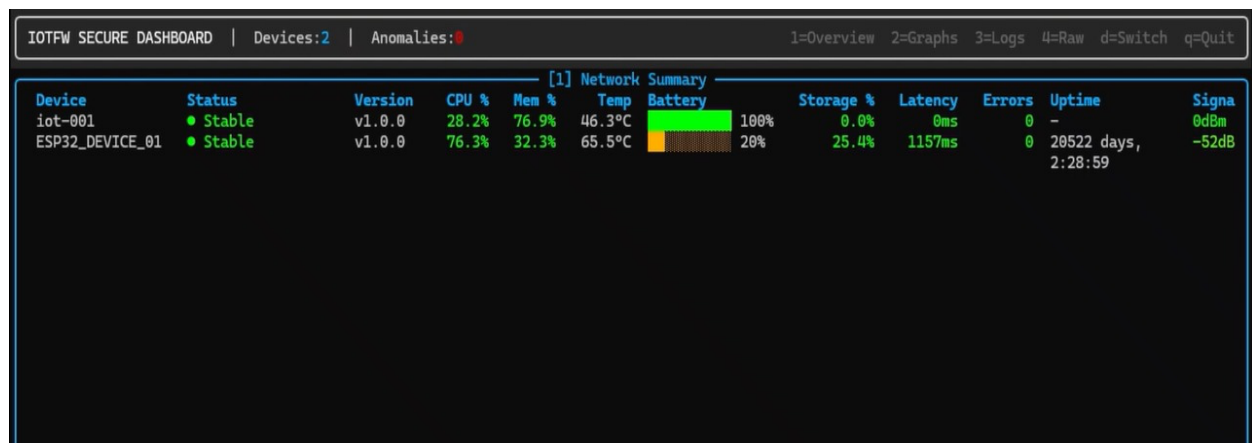


Figure: Dashboard capturing real time telemetry data

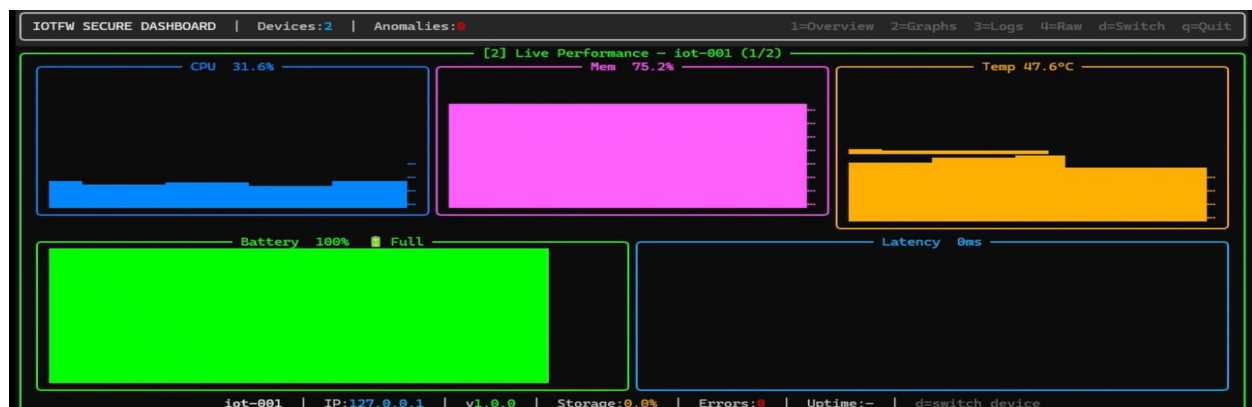


Figure: Graphical view of real time telemetry data

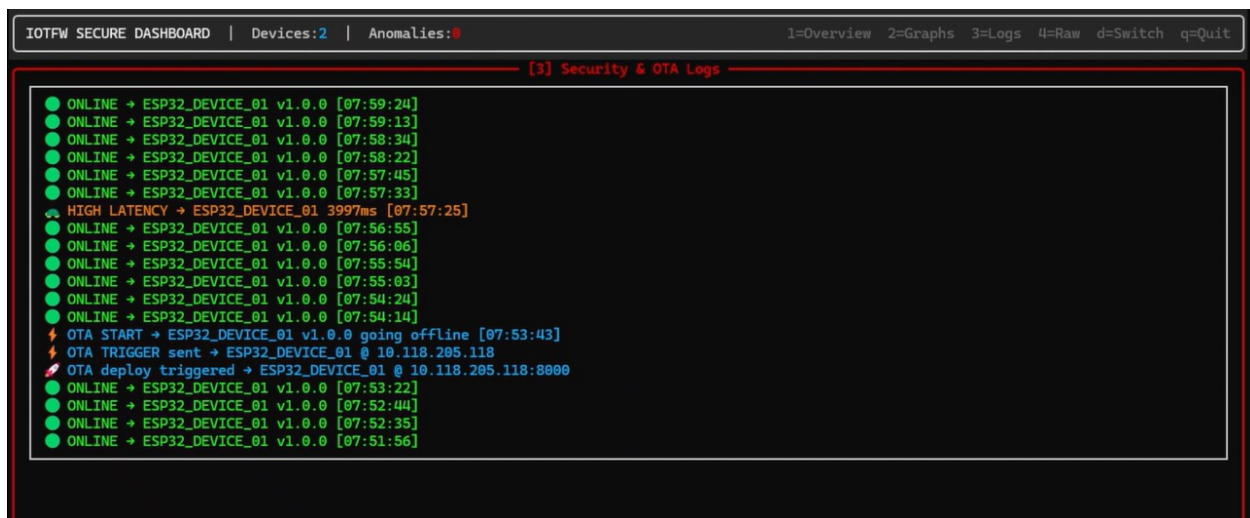


Figure: Alerts Dashboard

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Results

Layer 1 – Zero Trust Authorization: All four simulated attack scenarios were correctly detected and blocked. Hardware fingerprint tampering was detected and the device was quarantined with HTTP 400 returned. Counter replay attacks, signature forgery, and counter rollback attacks were all rejected with HTTP 401. No false positives were observed for legitimate device authentication. End to end authorization latency averaged 2,810 milliseconds across 50 sequential runs, broken down as follows: Phase 1 public key validation averaged 392.9 ms, Phase 2 hardware fingerprint binding averaged 413.92 ms, Phase 3 nonce generation and signature verification averaged 719.68 ms, Phase 5 compound payload verification averaged 785.61 ms, and RBAC trust score lookup averaged 353.51 ms.

Layer 2 – Lightweight Secure OTA Core: The decision based update strategy successfully differentiated between scenarios where delta patching was efficient and scenarios where full image replacement was required. The decision engine was evaluated across multiple firmware change patterns to validate the Decision Score formula as a reliable mechanism for adaptive payload selection. In a localized patch scenario where approximately 5% of windows were touched contiguously, metric values of SCR=0.06, FS=0.08, RC=0.10, and IR=0.15 produced DS=0.097, correctly selecting Delta · Instruction Aware. In a moderate divergence scenario with scattered fragmentation, SCR=0.25, FS=0.40, RC=0.30, and IR=0.20 produced DS=0.280, again correctly routing to the instruction aware delta strategy. Across all tested patterns the default threshold of 0.55 provided a clear and correct decision boundary, confirming that the four metric weighted formula is a valid quantitative proxy for delta applicability.

Layer 3 – Threshold Based Anomaly Detection: Under normal operating conditions, the system allowed updates to proceed without interruption and without introducing significant processing overhead, with telemetry evaluation completing in under 100 milliseconds. Under anomalous conditions including simulated high CPU usage, memory exhaustion, network instability, and multiple simultaneous anomalies, the system consistently identified threshold violations and blocked update delivery. Attack scenarios simulating malware like CPU and memory behavior and DDoS indicative network traffic were all correctly detected.

Layer 4 – Resilient Fail Safe Update Engine:

Metric	Value
Baseline Free Heap (Idle)	284.0KB
Minimum Free Heap (During OTA)	210.5KB
Streaming Crypto Buffer	4.0 KB
Active Attestation Task Overhead	3.5 KB
Threat Detection Latency	< 45 ms
Deterministic Rollback Time	420 ms

The active hardware attestation task detected the injected FREEZE execution fault in under 45 milliseconds and completed the full automated rollback in 420 milliseconds, restoring the device to the stable firmware partition before any permanent damage could occur

3.2 Research Findings

The combined results of the SecureTrustOTA framework yield several significant findings.

First, Zero Trust principles can be effectively applied to OTA firmware delivery for resource constrained IoT devices without requiring specialized trusted hardware extensions such as TPM or ARM TrustZone. The five phase authentication pipeline reliably detects hardware tampering, credential replay, signature forgery, and rollback attacks, providing a level of authorization assurance absent from existing frameworks such as Uptane, TUF, and ASSURED.

Second, decision based and instruction aware delta patching meaningfully improves OTA delivery efficiency compared to conventional approaches. By evaluating firmware structural characteristics before selecting an update strategy, the system avoids the instability of blindly applying byte level patches to structurally changed firmware.

Third, lightweight threshold based anomaly detection can provide effective real time pre update device health validation without the computational overhead of machine learning based approaches, making it viable for deployment in server managed IoT environments.

Fourth, active hardware attestation using a custom FSM and FreeRTOS task is both feasible and highly effective on constrained microcontrollers. The 3.5 KB memory overhead of the attestation task demonstrates that continuous behavioral profiling does not exhaust the limited SRAM of the ESP32, and the sub 45 millisecond detection latency represents an approximately 100 fold improvement over standard passive hardware watchdog timers.

Fifth, the end to end authorization latency of approximately 2,810 milliseconds, while not negligible, is operationally acceptable for OTA firmware updates, which are inherently infrequent operations where security assurance far outweighs the cost of per transaction overhead.

3.3 Discussion

SecureTrustOTA addresses limitations present in all existing OTA frameworks by operating across the full update lifecycle rather than focusing on a single dimension such as secure delivery or post update recovery. The layered architecture ensures that trust is enforced at authorization, delivery, pre update validation, and post installation execution stages simultaneously.

The Zero Trust Authorization Layer closes critical attack vectors present in federated trust systems by deriving trust scores exclusively from the server's own verification database, preventing compromised devices from self reporting fraudulent scores. The hardware fingerprinting mechanism introduces a detection capability entirely absent from existing OTA frameworks: physical device substitution and hardware level tampering detection.

The Lightweight Secure OTA Core's decision based update strategy resolves the false economy of indiscriminate delta patching, where structural firmware changes can produce patches larger or more unstable than a full replacement. Instruction aware normalization of address related values advances the practicality of delta updates in real deployments.

The Threshold Based Anomaly Detection Engine shifts the OTA security model from reactive to proactive, ensuring that devices in compromised or degraded operational states do not receive firmware that could exacerbate failures or propagate malicious code.

The Resilient Fail Safe Update Engine represents a significant architectural advancement over passive recovery mechanisms. By detecting and responding to stealthy execution faults before a

complete system crash, it protects edge nodes from malicious updates even in the absence of cloud connectivity, which is a critical requirement for Zero Trust deployments in industrial and remote environments.

Limitations include the use of SQLite as the backend database for the authorization layer, which may introduce write contention at high concurrency and should be replaced with PostgreSQL for large scale production deployments. The computational overhead of the five phase authentication pipeline on actual ESP32 hardware was not formally evaluated and represents an important direction for future validation. Static threshold values in the anomaly detection engine may require refinement for heterogeneous device populations, and hybrid approaches combining threshold based and lightweight machine learning methods warrant further investigation.

3.4 Summary of Each Student's Contribution

Dilmith K.P.H.S (IT21832130) was responsible for designing and implementing the Resilient Fail Safe Update Engine (Layer 4). This included the dual bank OTA architecture with air gapped SD card staging buffer, the streaming SHA 256 cryptographic verification pipeline, the custom Finite State Machine controlling the post update probation sequence, and the Active Hardware Attestation FreeRTOS task with automated rollback on the ESP32 platform.

Jayaweera N.S (IT22362780) led the design and implementation of the Zero Trust Authorization and Policy Enforcement Layer (Layer 1). This encompassed the five phase adaptive authentication pipeline, hardware bound device identity using MCU fingerprinting, dynamic RBAC and PBAC enforcement, firmware attestation, AES 256 CBC end to end encryption, JWT based authorization, and the nine category comprehensive audit logging architecture. Additionally contributed the secure web based administrator dashboard for OTA management with real time alerts.

Mandira U.L.L.K (IT22365996) This component forms the core firmware delivery layer of the SecureTrustOTA system, sitting between the zero trust authentication layer and the anomaly detection and fail safe layers contributed by other team members. It introduces adaptive payload selection, instruction aware delta generation, and peer assisted ESP NOW mesh distribution, directly determining what firmware is delivered, how it is packaged, and how efficiently it reaches each device. Without this layer, the broader system would lack the intelligence to right size update payloads, the cryptographic self description needed for multi format dispatch, and the scalable distribution mechanism required for co located deployments.

P.A.S.N. Perera (IT22307262) developed the Threshold Based Anomaly Detection Engine (Layer 3). This included the telemetry collection system, threshold definition and adaptive thresholding logic, real time anomaly detection and alert generation, and a lightweight monitoring dashboard for observing ESP32 performance metrics including RAM usage during OTA operations.

CHAPTER 4: CONCLUSION

This dissertation presented SecureTrustOTA, a multi layered, resilient, and trust based OTA firmware update framework for IoT devices. The framework was designed to address the fundamental limitations of existing OTA systems, which focus narrowly on secure delivery while neglecting continuous device trust verification, intelligent update selection, pre update health validation, and active post installation resilience.

The four layer architecture collectively enforces security, efficiency, and resilience across every stage of the firmware update lifecycle. The Zero Trust Authorization Layer ensures that only cryptographically verified, hardware consistent, and contextually trusted devices receive firmware. The Lightweight Secure OTA Core ensures that firmware is delivered in the most efficient and stable form through decision based and instruction aware patching. The Threshold Based Anomaly Detection Engine ensures that devices in compromised or degraded states are proactively excluded from update operations. The Resilient Fail Safe Update Engine ensures that even if a malicious or faulty firmware payload passes prior validation, the edge node can autonomously detect and recover from execution faults before permanent damage occurs.

Experimental evaluation confirmed that the framework effectively detects and blocks all tested attack vectors, achieves fault detection in under 45 milliseconds with a memory overhead of only 3.5 KB, and maintains operationally acceptable authorization latency for an infrequent, high stakes operation such as firmware deployment.

Future work should focus on evaluating the full pipeline on actual resource constrained hardware, integrating post quantum cryptographic schemes, extending the anomaly detection engine with hybrid threshold and lightweight machine learning models, migrating to a production grade database backend, and exploring hierarchical peer assisted mesh distribution architectures for massive IoT deployments.

SecureTrustOTA represents a meaningful advancement in the state of OTA security for IoT, offering a practical, scalable, and comprehensive solution for organizations seeking to protect their connected device ecosystems against modern firmware targeting threats.

REFERENCES

- [1] K. Sabarishraj et al., "Efficient Implementation of Firmware Over the Air Update using Raspberry Pi 4 and STM32F103C8T6," in 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, 2024, pp. 1–6.
- [2] S. El Jaouhari and E. Bouvet, "Secure firmware over the air updates for IoT: Survey, challenges, and discussions," *Internet of Things*, vol. 18, 100508, 2022.
- [3] F. Meneghello et al., "IoT: Internet of Threats? A Survey of Practical Security Vulnerabilities in Real IoT Devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8182–8201, Oct. 2019.
- [4] Espressif Systems, "ESP IDF programming guide," Version 5.2, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- [5] National Institute of Standards and Technology (NIST), "Zero trust architecture," NIST Special Publication 800 207, Aug. 2020.
- [6] N. Asokan et al., "ASSURED: Architecture for secure software update of realistic embedded devices," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2290–2300, Nov. 2018.
- [7] J. L. Hernández Ramos et al., "Updating IoT devices: Challenges and potential approaches," in *Proc. Global Internet of Things Summit (GloTS)*, Dublin, Ireland, Jun. 2020, pp. 1–6.
- [8] C. Liu et al., "Dissecting zero trust: Research landscape and its implementation in IoT," *Cybersecurity*, vol. 7, no. 1, p. 20, Mar. 2024.
- [9] FreeRTOS, "FreeRTOS Reference Manual: API Functions and Configuration Options," 2025. [Online]. Available: <https://www.freertos.org>
- [10] Microchip Technology Inc., "ATECC608A CryptoAuthentication device summary datasheet," 2020.
- [11] National Institute of Standards and Technology (NIST), "Platform Firmware Resiliency Guidelines," NIST Special Publication 800 193, May 2018.
- [12] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018.